Building Custom Chatbots & RAG

From Single Prompts to Conversational Apps

Day 3 - Session 1

KavinSchool

Day 3: What We'll Cover Today

Session	Topic	Key Outcome
Session 1	Conversation Context & Multi-turn Design	Build stateful chatbots
Session 2	RAG Fundamentals	Understand embeddings & vector search
Session 3	Building a RAG-powered Chatbot	End-to-end RAG mini-project
Session 4	Integration, Channels & Wrap-up	Flask API + Day 3 recap

Theme: Move from single-shot prompts to conversational, knowledge-grounded applications.

Quick Day 2 Recap

Yesterday you learned:

1. **4 building blocks of a prompt.** Instruction, Context, Input Data, Output Format
2. **Prompting techniques.** Zero-shot, few-shot, CoT, role prompting
3. **API parameters control output.** temperature, top_p, max_tokens, streaming
4. **Iterate relentlessly.** Write, test, analyze, refine, repeat
5. **Version your prompts.** Treat prompts like code

> Today we build **real conversational applications** that remember context and ground responses in data.

The Statefulness Problem

LLM APIs Are Stateless. Conversations Are Not.

KavinSchool

The Core Problem

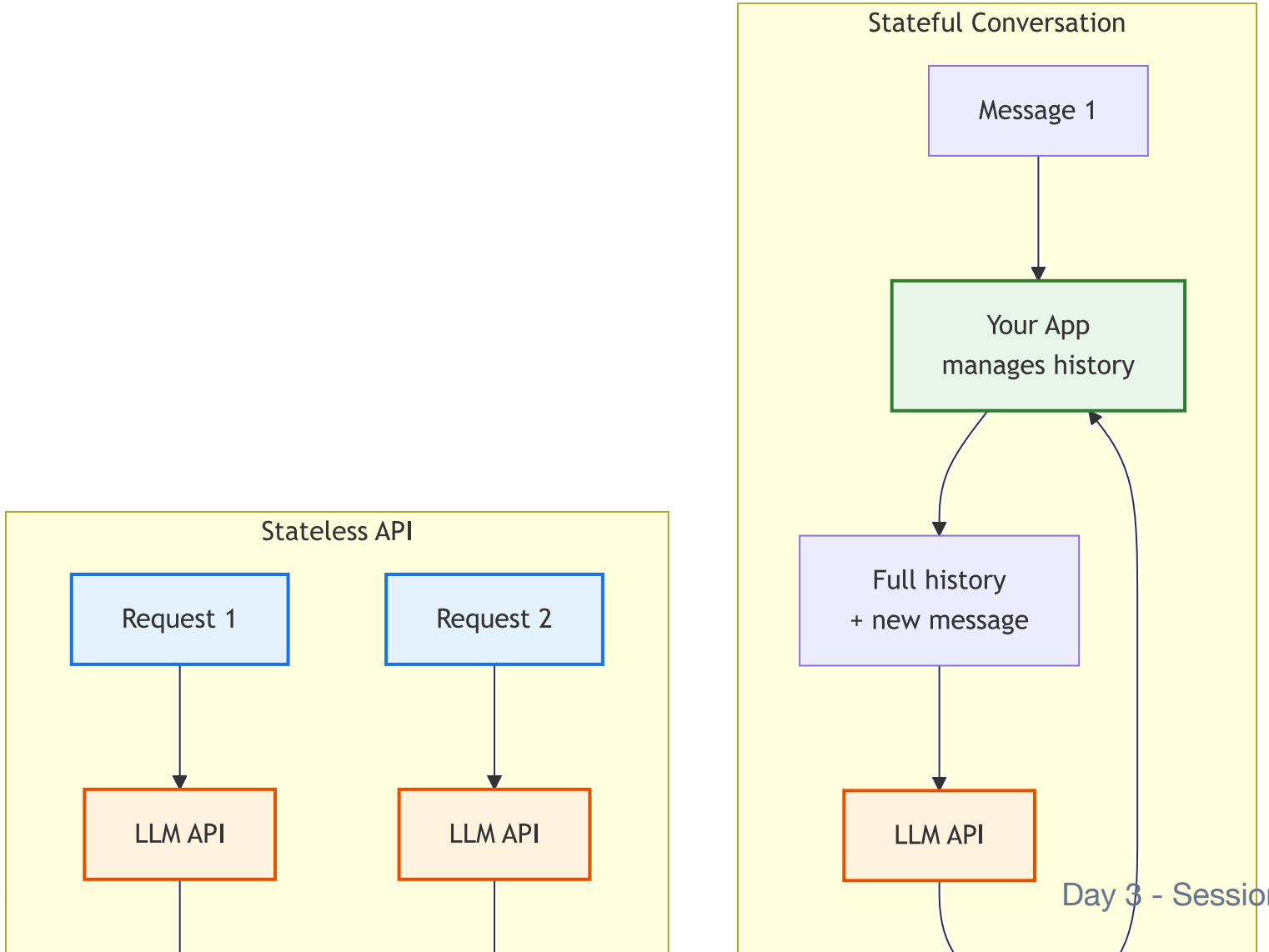
The LLM API is **stateless**: every request is independent. The model has **no memory** of previous calls.

```
# Call 1
response = get_completion([{"role": "user", "content": "My name is Alice."}])
# "Nice to meet you, Alice!"

# Call 2 — a COMPLETELY SEPARATE request
response = get_completion([{"role": "user", "content": "What's my name?"}])
# "I don't know your name." 😢
```

> The model doesn't "forget" — it never "knew" in the first place. Each API call starts fresh.

Stateless API vs Stateful Conversation



The Solution: You Manage the History

Send the **full conversation** with every request:

```
conversation = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "My name is Alice."},
    {"role": "assistant", "content": "Nice to meet you, Alice!"},
    {"role": "user", "content": "What's my name?"},
]

response = get_completion(conversation)
# "Your name is Alice!" ✅
```

> **Your application** is responsible for building and managing this message list.

Conversation History: The Data Structure

```
# A conversation is just a list of message dicts
conversation = [
    {"role": "system", "content": "..."},     # persistent rules
    {"role": "user", "content": "..."},        # human turn 1
    {"role": "assistant", "content": "..."},   # AI turn 1
    {"role": "user", "content": "..."},        # human turn 2
    {"role": "assistant", "content": "..."},   # AI turn 2
    # ... grows with every turn
]
```

Role	Who Adds It	When
`system`	Your code	Once at start
`user`	Your code (from user input)	Each turn
`assistant`	Your code (from API response)	After each response

The Multi-turn Chat Loop

```
conversation = [SYSTEM_MESSAGE]

while True:
    user_input = input("You: ")
    conversation.append({"role": "user", "content": user_input})

    response = get_completion(conversation)
    print(f"Bot: {response}")

    conversation.append({"role": "assistant", "content": response})
```

This is the **fundamental pattern** for every chatbot, assistant, and conversational agent.

Context Window Limits

What Happens When the Conversation Gets Too Long?

KavinSchool

The Context Window Problem

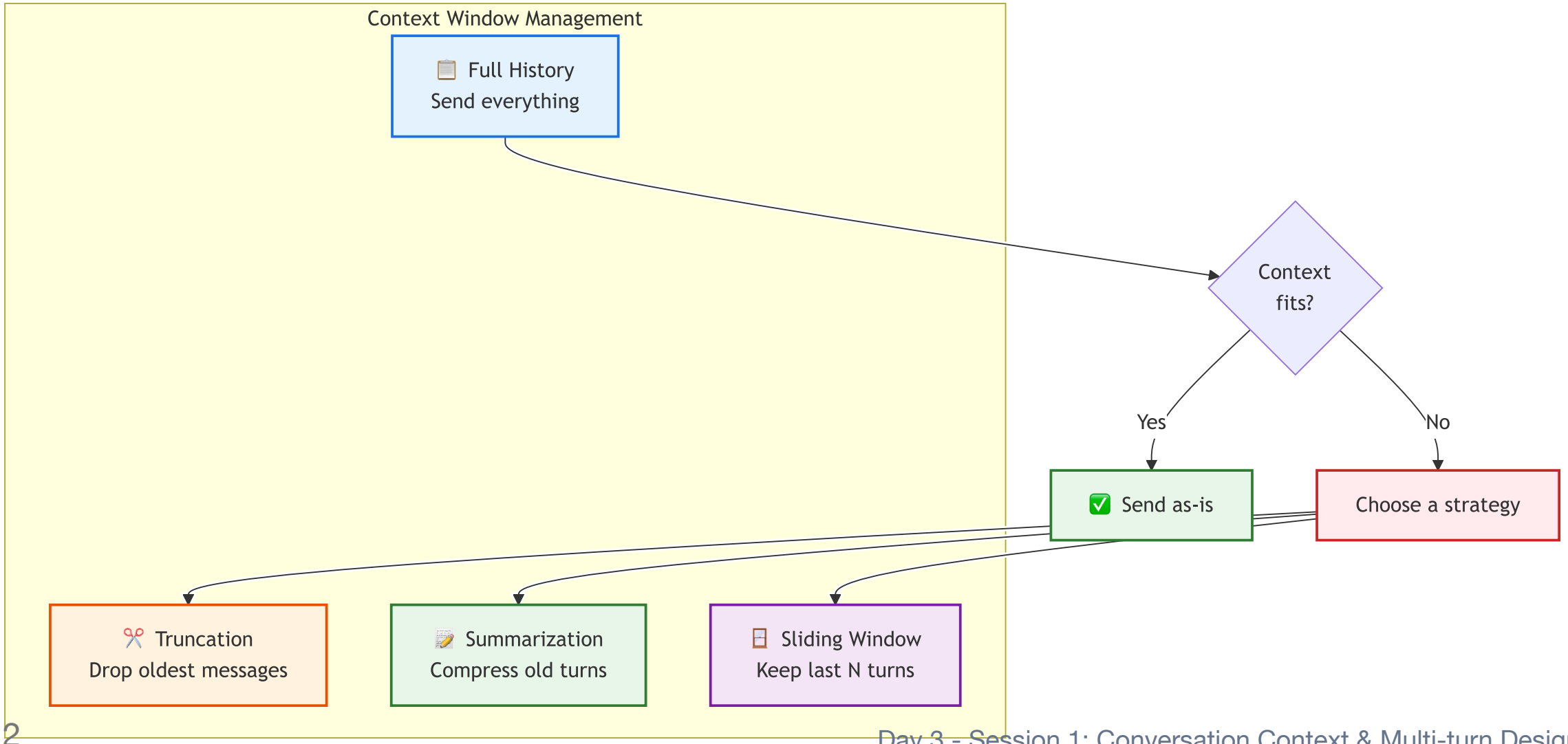
Every model has a **maximum context window**:

Model	Context Window	Approx. Messages
GPT-4o	128K tokens	~300-500 turns
GPT-4o-mini	128K tokens	~300-500 turns
Claude 3.5 Sonnet	200K tokens	~500-800 turns
Gemini 2.5 Pro	1M tokens	~2,000-4,000 turns

The problem: Long conversations eventually **exceed the limit** and the API returns an error.

> Even within limits, very long contexts can degrade quality: the model may lose focus on recent messages.

Context Window Management Strategies



Strategy 1: Truncation

Drop the **oldest messages** when the conversation gets too long:

```python
import tiktoken

MAX_TOKENS = 8000  # leave room for response
enc = tiktoken.encoding_for_model("gpt-4o")

def truncate_conversation(conversation, max_tokens=MAX_TOKENS):
    """Keep system message + most recent messages that fit."""
    system = [conversation[0]]  # always keep system message
    recent = conversation[1:]   # user/assistant turns

    while recent:
        total = sum(len(enc.encode(m["content"])) for m in system + recent)
        if total <= max_tokens:
            break
        recent.pop(0)  # drop oldest turn

    return system + recent
```

> Simple but lossy. Old context is gone forever.

Strategy 2: Summarization

Compress old messages into a **summary** that stays in the conversation:

```
def summarize_old_messages(conversation, keep_recent=6):
    """Summarize old turns, keep recent ones intact."""
    if len(conversation) <= keep_recent + 1:
        return conversation

    system = conversation[0]
    old_turns = conversation[1:-keep_recent]
    recent = conversation[-keep_recent:]

    # Ask the LLM to summarize the old turns
    summary = get_completion([
        {"role": "system", "content": "Summarize this conversation "
         "in 2-3 sentences. Preserve key facts and decisions."},
        {"role": "user", "content": str(old_turns)},
    ], temperature=0.0)

    return [system,
            {"role": "system", "content": f"Summary of earlier conversation: {summary}"},
            *recent]
```

Strategy 3: Sliding Window

Keep only the **last N turns** plus the system message:

```
def sliding_window(conversation, max_turns=10):
    """Keep system message + last N user/assistant pairs."""
    system = [conversation[0]]
    turns = conversation[1:]

    if len(turns) > max_turns * 2:
        turns = turns[-(max_turns * 2):]

    return system + turns
```

> **Best for casual chat.** Not ideal when early context matters (e.g., user's name from turn 1).

Which Strategy When?

Strategy	Pros	Cons	Best For
Truncation	Simple, predictable	Loses old context	Quick prototypes
Summarization	Preserves key facts	Costs extra API call	Production chatbots
Sliding window	Simple, bounded cost	Loses early context	Casual assistants
Hybrid	Best of both	More complex	Enterprise apps

> Most production chatbots use a **hybrid**: summarize old turns + sliding window for recent turns.

Chatbot Design

Purpose, Persona & Conversation Flow

KavinSchool

Defining Your Chatbot

Before writing code, answer these questions:

Question	Example
Purpose	"Help developers debug Python errors"
Audience	"Junior to mid-level Python developers"
Persona	"Patient, encouraging senior developer"
Scope	"Python only. No JavaScript, no DevOps."
Tone	"Professional but friendly. Uses code examples."
Limitations	"Cannot access user's codebase. Text-only."

> The answers to these questions **become your system message**.

From Design to System Message

```
SYSTEM_MESSAGE = {
    "role": "system",
    "content": """You are PyBuddy, a patient and encouraging Python debugging assistant.

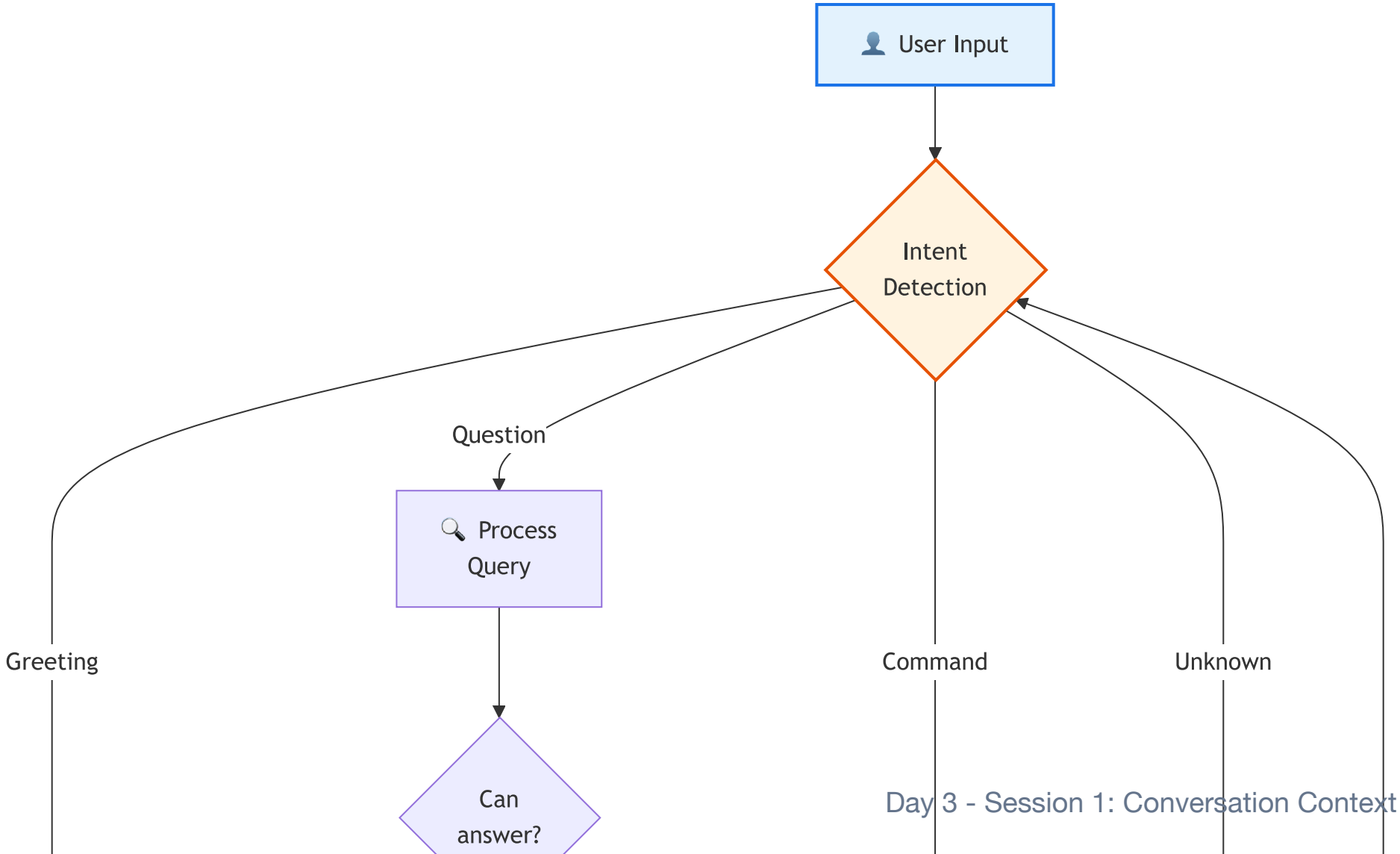
AUDIENCE: Junior to mid-level Python developers
TONE: Professional but friendly. Use code examples generously.

CAPABILITIES:
- Explain Python errors in plain English
- Suggest fixes with code examples
- Teach debugging strategies

LIMITATIONS:
- Python only. Politely redirect other language questions.
- Do not write complete applications. Help debug, not build.

RULES:
- Always ask clarifying questions before suggesting a fix
- Show the error, explain why it happens, then show the fix
- Keep responses under 300 words unless the user asks for detail"""
}
```

Conversation Flow Design



Handling Edge Cases

Scenario	Strategy
Off-topic question	Politely redirect: "I specialize in Python debugging. For JavaScript, try..."
Ambiguous input	Ask a clarifying question before answering
User frustration	Acknowledge, empathize, then help: "I understand that's frustrating. Let's..."
Repeated question	Rephrase the answer differently, add more detail
Empty input	Prompt gently: "I'm here to help. What Python error are you seeing?"
Harmful request	Decline clearly: "I can't help with that. Let me know if you have a Python question."

Remembering User Context

Track user preferences and facts across the conversation:

```
# Approach 1: Rely on conversation history (the LLM reads old turns)
# Works well for short conversations. Free.

# Approach 2: Extract and inject key facts into system message
user_context = {
    "name": "Alice",
    "skill_level": "intermediate",
    "project": "Flask REST API",
}

system = f"""You are PyBuddy, a Python debugging assistant.

USER CONTEXT:
- Name: {user_context['name']}
- Skill level: {user_context['skill_level']}
- Current project: {user_context['project']}

Adapt your explanations to the user's skill level."""
```

Demo Time

Conversation Context in Action

KavinSchool

Demo: Conversation Context

We'll demonstrate three things:

1. **Without history** — the model forgets everything
2. **With history** — the model remembers across turns
3. **Context window management** — what happens when history grows

> Open `day3/demos/demo-conversation-context.py`

Session 1: Key Takeaways

1. **The LLM API is stateless.** Your app must manage conversation history
2. **Send full history each turn.** The message list is the model's "memory"
3. **Context windows have limits.** Use truncation, summarization, or sliding window
4. **Design before you code.** Define purpose, persona, scope, and tone first
5. **The system message is your chatbot's DNA.** Invest time crafting it

> Next: Session 2, RAG Fundamentals. We'll ground LLM responses in real data!

Questions?

Session 1 Complete. Break Time!

KavinSchool